

Procesamiento Digital de Señales

Proyecto 2: Integración de métodos de PDS en C++

David Monge Naranjo — 2019158668
Alejandro Araya Núñez — 2015041642
Daniel González Vargas — 2019081347

1. Reverberador

Para la implementación del reverberador, se realizaron 2 versiones, una con el modelo facilitado por el profesor y una variante de este usando un filtro paso bajos en serie con el retardador de la señal de salida. Para ambas implementaciones se realizó un filtro en el tiempo con ecuaciones de diferencias.

1.1. Reverberador simple

Este sigue la siguiente ecuación:

$$y[n] = x[n] + \alpha y[n - \tau] \quad (1)$$

El valor de α puede variar desde 0 hasta 1, donde este representa la atenuación de la réplica percibida por la reverberación, siendo 0 una atenuación inmediata y 1 una atenuación nula. Por otra parte, el valor de τ representa el retardo o distancia entre réplicas, siendo utilizado como cantidad de muestras, este se configura para variar entre 1024 y 94208, en el mínimo se tiene una distancia entre réplicas de aproximadamente 0.02 segundos y en el máximo se tiene una distancia de alrededor de 2 segundos.

1.2. Reverberador filtrado

En este caso, se utilizó el reverberador de la Fig.1, donde para este se puede configurar los mismos parámetros del reverberador anterior y también se puede cambiar de posición el polo del filtro

paso bajos usado, de esta forma cambiando la frecuencia de corte.

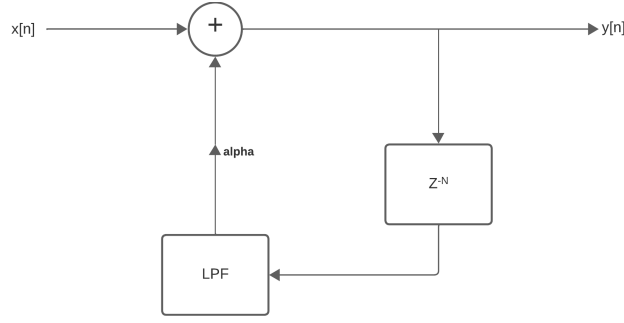


Figura 1: Diagrama de bloques usado para el segundo reverberador

El valor del polo del filtro paso bajos se puede colocar en un rango desde 0 hasta 1.

2. Filtro Peine $k \cdot 60\text{Hz}$

Para el filtro peine se basó en la siguiente ecuación:

$$y[n] = K \cdot (x[n] - x[n - L]) \quad (2)$$

Este genera L ceros en bandas espaciadas de 0 a 2π , para lograr el efecto deseado, se calcula L como $f_s s / f_0$. Con $f_s = 48\text{kHz}$ se obtiene $L = 800$. Con esto sin embargo las bandas son bastantes anchas, por lo que se añadieron polos para hacer más angostas las bandas, se escogió un valor de $r = 0,8$. A partir de esto se calculo el valor de K para obtener una ganancia de 1 en 30Hz, esto luego es periódico en cada valle. Con lo que se obtiene la ecuación del filtro implementado en el tiempo.

$$ey[n] = K \cdot (x[n] - x[n - L]) + r \cdot y[n - L] \quad (3)$$

Resolviendo con transformada Z se obtiene el valor de K

$$r = 0,8 \implies K = 0,9 \quad (4)$$

Se implementa con un buffer circular con 800 de tamaño para obtener los valores desfasados tanto de x como de y .

Se muestra en la figura 3, la respuesta en magnitud y fase del filtro.

3. Ecualizador de 20 bandas

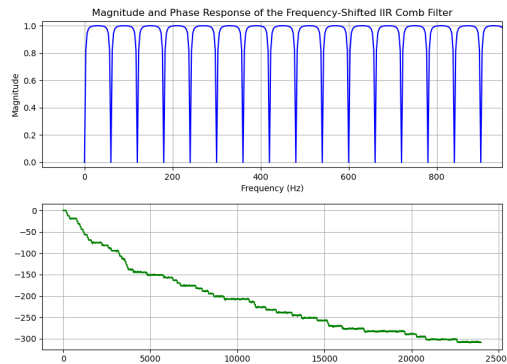


Figura 2: Respuesta en magnitud y fase del filtro peine

3.1. Interpolación

La interpolación se realiza utilizando la librería GSL (GNU Scientific Library). En este caso se utiliza el interpolador Akima el cual genera cambios bruscos más bruscos que el interpolador cúbico a cambio de mantener la interpolación en un rango cercano a los valores de muestra.

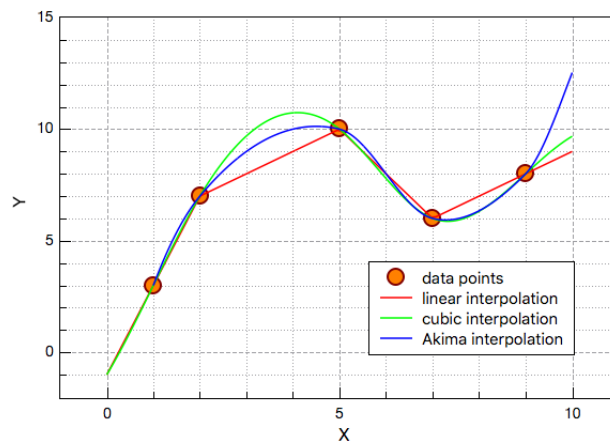


Figura 3: Comparativa entre funciones de interpolación

3.2. Implementación

Para la implementación se utiliza transformada rápida de Fourier, utilizando el método de solapamiento y suma. A partir de los resultados de los valores de la función de interpolación se pasa esto a la clase `filter` los valores de H_w . Ya que se utiliza una magnitud igual a los valores obtenidos de interpolación, estos se asignan a la parte real de H_w , lo cual genera que el sistema tenga una fase de 0.

Al ser un filtro con fase nula, tiene como implicación que no es un filtro causal, por lo que depende de muestras futuras. Para solucionar esto, se utiliza el bloque de entrada actual como muestra futura y el bloque de entrada pasado como muestra actual por lo que el filtro tiene un retraso de un bloque.